

字典(dict)和集合(set, frozenset)

邓擎琮
北京师范大学

典型应用

- 字典：通讯录
- 集合：分析两人的共同好友

```
print('1: 查询联系人信息; 2: 新建联系人; 3: 删除联系人; 4: 退出程序')
```

```
li = dict()
```

```
while True:
```

```
    try:
```

```
        taskId = input('请输入任务代码1/2/3/4: ')
```

```
        if taskId not in ['1', '2', '3', '4']:
```

```
            raise
```

```
        if taskId == '1':
```

```
            name = input('请输入要查询的联系人姓名: ')
```

```
            print(f'电话: {li[name]}')
```

```
        elif taskId == '2':
```

```
            name = input('请输入新建联系人姓名: ')
```

```
            phone = input('请输入新建联系人电话: ')
```

```
            li[name] = phone
```

```
        elif taskId == '3':
```

```
            name = input('请输入要删除的联系人姓名: ')
```

```
            del li[name]
```

```
        else:
```

```
            break
```

```
    except RuntimeError:
```

```
        print('输入错误, 字符不是1-4, 请重新输入!')
```

```
    except KeyError:
```

```
        print('要操作的联系人不存在!')
```

----欢迎使用通讯录程序，使用方法如下：-----

1: 查询联系人信息； 2: 新建联系人； 3: 删除联系人； 4: 退出程序

请输入任务代码1/2/3/4: 2

请输入联系人姓名: 张三

请输入联系人电话，地址: 138111, 学15-303

请输入任务代码1/2/3/4: 2

请输入联系人姓名: 李四

请输入联系人电话，地址: 138222, 学16-204

请输入任务代码1/2/3/4: 1

请输入要查询的联系人姓名: 张三

电话: 138111 地址: 学15-303

请输入任务代码1/2/3/4: 4

In [2]: li

Out[2]: {'张三': ['138111', '学15-303'], '李四': ['138222', '学16-204']}

In [3]: li

Out[3]: {'张三': ['138111', '学15-303'], '李四': ['138222', '学16-204']}

In [4]: wu

Out[4]: {'李四': ['138222', '学16-204'], '王天': ['159000', '学13-101']}

In [5]: s1=set(li);s2=set(wu)

In [6]: print(s1,s2)

{'张三', '李四'} {'李四', '王天'}

In [7]: s1&s2

Out[7]: {'李四'}

In [8]: s1|s2

Out[8]: {'张三', '李四', '王天'}

In [9]: s1-s2

Out[9]: {'张三'}

字典和集合相同点

- 不能按位置索引取值
- 不重复：字典的**key**不重复，集合元素不重复

不可变对象

bool, int, float, complex,
str, tuple, frozenset

list, dict, set



`TypeError: unhashable type:`

字典和集合常用共有操作

- 内置函数len(), max(), min(), sum(), sorted(), list().....

```
>>> d1={1:'a',2:'b'}
```

```
>>> len(d1),min(d1),sum(d1)
(2, 1, 3)
```

```
>>> sorted(d1)
[1, 2]
```

```
>>> list(d1)
[1, 2]
```

```
>>> s1={5,3,1}
```

```
>>> len(s1),max(s1),sum(s1)
(3, 5, 9)
```

- in, not in

```
>>> 1 in d1
True
```

```
>>> 'a' in d1
False
```

```
>>> 3 in s1
True
```

```
>>> 5 not in s1
False
```

常用共有操作3

■ for循环操作

```
>>> d2={1:'boy',2:'girl'}  
>>> for i in d2:  
    print(i)
```

1

2

```
>>> s1={5,3,1}  
>>> for ss in s1:  
    print(ss)
```

1

3

5

常用共有操作2

■ 比较运算: ==, !=

```
>>> d1={1:'a',2:'b'}
```

```
>>> d2={1:'boy',2:'girl'}
```

```
>>> d1==d2
```

```
False
```

```
>>> s1={5,3,1}
```

```
>>> s2={1,3}
```

```
>>> s1!=s2
```

```
True
```

```
>>> d1>d2
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#107>", line 1, in <module>
```

```
    d1>d2
```

```
TypeError: unorderable types: dict() > dict()
```

```
>>> s1>s2
```

```
True
```

字典

■ 字典的定义或创建:

- {key1: value1[,key2:value2,...,keyn:valuen] }
- {} 或 dict(): 创建一个空字典
- dict(mapping): 从一个字典对象创建一个新的字典
- dict(iterable): 使用可迭代对象创建字典
- dict(**kwargs): 使用关键字参数创建字典

```
>>> d1={ }
>>> d1={1:'apple',2:'banana'}
>>> print(d1,type(d1))
{1: 'apple', 2: 'banana'} <class 'dict'>

>>> {'fruit': d1}
{'fruit': {1: 'apple', 2: 'banana'}}
```

字典

■ 字典的定义或创建:

- {key1: value1[,key2:value2,...,keyn:valuen] }
- {} 或 dict(): 创建一个空字典
- dict(mapping): 从一个字典对象创建一个新的字典
- dict(iterable): 使用可迭代对象创建字典
- dict(**kwargs): 使用关键字参数创建字典

```
>>> d1={ }
```

```
>>> d1={1:'apple',2:'banana'}
```

```
>>> {d1:'fruit'}
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#32>", line 1, in <module>  
    {d1:'fruit'}
```

```
TypeError: unhashable type: 'dict'
```

字典的创建举例

```
>>> d2=dict(d1)
>>> print(id(d1), id(d2), d2)
56129608 56114824 {1: 'apple', 2: 'banana'}
```

长度必须为2



```
>>> dict((( 'sn', '201601'), ('name', 'Jenny')))
{'sn': '201601', 'name': 'Jenny'}
```

```
>>> dict([('sn', '201601'), ['Phone', [138, 5880]]])
{'sn': '201601', 'Phone': [138, 5880]}
```

```
>>> dict(name='Bob', age=40)
{'name': 'Bob', 'age': 40}
```

使用给定的键新建字典

■ **fromkeys(iterable, value=None)**

```
>>> {}.fromkeys(['name', 'age'])  
{'name': None, 'age': None}
```

```
>>> dict.fromkeys(['name', 'age'])  
{'name': None, 'age': None}
```

```
>>> dict.fromkeys(['name', 'age'], 'unknown')  
{'name': 'unknown', 'age': 'unknown'}
```

通过复制创建字典

- **copy**方法返回一个具有相同键/值对的新的字典

```
>>> d={'spam':20,'cheese':30}
>>> d1=d.copy()
>>> print(d1, '\n', id(d), id(d1))
{'spam': 20, 'cheese': 30}
49341280 49340992
```

字典的操作

- 查看
- 字典是可变对象：
 - 增
 - 改
 - 删

字典的查看操作

1. **d[key]**: 返回key的value, 如果key不存在将导致KeyError

```
>>> d={'spam':20, 'cheese':30}
```

```
>>> d['cheese']
```

```
30
```

```
>>> d[1]
```

```
Traceback (most recent call last):
```

```
File "<pyshell#13>", line 1, in <module>
```

```
d[1]
```

```
KeyError: 1
```

2. **get(key[,v])**: 返回key对应的value, 如果key不存在, 不报错, 而是返回v (默认返回None)

```
>>> d.get('spam')
```

```
20
```

```
>>> d.get(1)
```

```
>>> d.get(1, '空')
```

```
'空'
```


字典的查看操作

3. **keys()**、**values()**、**items()**分别返回字典中所有的键、值、键/值对

```
>>> d={'spam':20,'cheese':30}
>>> d.keys()
dict_keys(['spam', 'cheese'])
>>> d.values()
dict_values([20, 30])
>>> d.items()
dict_items([('spam', 20), ('cheese', 30)])
```

```
>>> for k in d.keys():
        print(k)
```

```
>>> list(d.keys())
['spam', 'cheese']
```

```
spam
cheese
```

字典的增、改操作

d[key]=value : 如果key存在, 则修改key的值为value, 如果key不存在则添加key:value对

```
>>> d['spam']=50
>>> d
{'spam': 50, 'cheese': 30}
```

```
>>> d['milk']=10
>>> d
{'spam': 50, 'cheese': 30, 'milk': 10}
```

字典的增、查看操作

setdefault(key[,value]): 如果key不存在, 添加key:value (默认为None) 对; 如果key存在, 则和get方法类似, 能够获得key相关联的value。

d[k] 和 d[k]=v

```
>>> d={'spam':20,'cheese':30}
>>> d.setdefault('cheese')
30
```

```
>>> d.setdefault('egg')
>>> d
{'spam': 20, 'cheese': 30, 'egg': None}
```

```
>>> d.setdefault('milk',10)
10
>>> d
{'spam': 20, 'cheese': 30, 'egg': None, 'milk': 10}
```

字典的改（更新）操作

- **update(other)**: 利用一个dict/iterable更新字典

```
>>> d={'spam': 20, 'cheese': 30}
>>> t={'spam': '二十', 1: '其他'}
>>> d.update(t)
>>> d
{'spam': '二十', 'cheese': 30, 1: '其他'}
```

长度必须为2



```
>>> d.update(['cheese', '三十'],)
>>> d
{'spam': '二十', 'cheese': '三十', 1: '其他'}
```

字典的删操作

1. **clear**方法清除字典中所有的项（即键/值对），变为空字典

```
>>> d={'spam':20,'cheese':30}
>>> d.clear()
>>> d
{}

```

字典的删操作

2. **del d[key]** : 删除key所在元素，如果key不存在将导致KeyError

```
>>> d={'spam':20, 'cheese':30}
```

```
>>> del d['spam']
```

```
>>> d
```

```
{'cheese': 30}
```

```
>>> del d[1]
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#21>", line 1, in <module>
```

```
    del d[1]
```

```
KeyError: 1
```

字典的删操作

3. **pop(k)**: 返回k对应的value, 并删除对应元素, 如果k不存在, 将导致KeyError

pop(k,v): 同上, 但是k不存在时返回v

```
>>> d={'spam':20, 'cheese':30}
```

```
>>> d.pop('spam')
```

```
20
```

```
>>> d
```

```
{'cheese': 30}
```

```
>>> d.pop(1)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#167>", line 1, in <module>
```

```
    d.pop(1)
```

```
KeyError: 1
```

```
>>> d.pop(1, '无')  
'无'
```

字典的删操作

4. **popitem()**: 弹出字典中最后一项(key, value)组成的元组, 如果字典为空, 则将导致KeyError

```
>>> d={'spam':20, 'cheese':30}
```

```
>>> d.popitem()
```

```
('cheese', 30)
```

```
>>> d.popitem()
```

```
('spam', 20)
```

```
>>> d.popitem()
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#172>", line 1, in <module>
```

```
    d.popitem()
```

```
KeyError: 'popitem(): dictionary is empty'
```


集合

- 可变集合set
- 不可变集合frozenset

集合的定义

- $\{ x1[,x2,x3,...,xn] \}$: set对象
- `set()`: 空set对象
- `set(iterable)`: 可迭代对象 → set对象
- `frozenset()`: 空frozenset对象
- `frozenset(iterable)`: 可迭代对象 → frozenset对象

```
>>> a={1, 1, 2, 2, 'a', True, 2.5}
```

```
>>> print(a, type(a))
```

```
{1, 2, 2.5, 'a'} <class 'set'>
```

```
>>> {'a', [1,2]}
```

```
Traceback (most recent call last):
```

```
File "<pyshell#30>", line 1, in <module>
```

```
    {'a', [1,2]}
```

```
TypeError: unhashable type: 'list'
```

集合的定义举例

```
>>> set('你好!!!')
```

```
{'你', '好', '!'}
```

```
>>> frozenset(('a', 'a', 'b', 'c'))
```

```
frozenset({'c', 'a', 'b'})
```

```
>>> set([1, 1, 2, 2, 3])
```

```
{1, 2, 3}
```

```
>>> set({'a': 'apple', 'b': 'banana'})
```

```
{'a', 'b'}
```

```
>>> frozenset({'a': 'apple', 'b': 'banana'})
```

```
frozenset({'a', 'b'})
```

集合的基本操作

1. 比较运算：相等、子集和超集($==$, $!=$, $<$, $<=$, $>$, $>=$)

```
>>> s1={1,2,3}
```

```
>>> s2={3,2,1}
```

```
>>> s3={1,2}
```

```
>>> s1==s2
```

```
True
```

```
>>> s3<s1
```

```
True
```

```
>>> s3<=s1
```

```
True
```

```
>>> s1>=s2
```

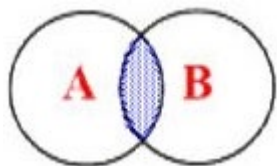
```
True
```

```
>>> s1>s3
```

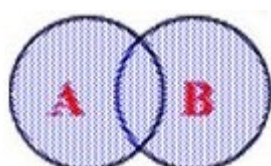
```
True
```

集合的基本操作

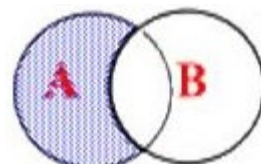
2. 交、并、差、对称差



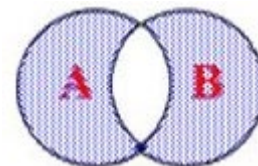
交: $\&$



并: $|$



差: $-$



对称差: $\wedge$

```
>>> s1={1, 2, 3}
```

```
>>> s2={2, 3, 4}
```

```
>>> s1 | s2
```

```
{1, 2, 3, 4}
```

```
>>> s1 & s2
```

```
{2, 3}
```

```
>>> s1 - s2
```

```
{1}
```

```
>>> s1 ^ s2
```

```
{1, 4}
```

```
>>> s3={1, 2, 5}
```

```
>>> s1 | s2 | s3
```

```
{1, 2, 3, 4, 5}
```

```
>>> s1 & s2 & s3
```

```
{2}
```

```
>>> s1 - s2 - s3
```

```
set()
```

集合的基本操作

3.集合的方法:

s不改变

方法	说明
<code>s.copy()</code>	复制集合
<code>s.isdisjoin(s2)</code>	如果s和s2交集为空, 返回True, 否则False
<code>s.issubset(s2)</code>	如果s是s2的子集, 返回True, 否则False
<code>s.issuperset(s2)</code>	如果s是s2的超集, 返回True, 否则False
<code>s.union(s2,...)</code>	s,s2,...的并集, 相当于 $s s2 ...$
<code>s.intersection(s2,...)</code>	s,s2,...的交集, 相当于 $s\&s2\&...$
<code>s.difference (s2,...)</code>	s,s2,...的差集, 相当于 $s-s2-...$
<code>s.symmetric_difference (s2)</code>	s,s2的对称差集, 相当于 s^s2

可变集合set的方法

■ set自有方法

$s1 = \{1, 2, 3\}; \quad s2 = \{2, 3, 4\}$

方法	说明	示例
<code>s.update(s2,...)</code>	$s = s2 \dots$	<code>s1.update(s2)</code> $\#s1 = \{1, 2, 3, 4\}$
<code>s.intersection_update(s2,...)</code>	$s \&= s2 \& \dots$	<code>s1.intersection_update(s2)</code> $\#s1 = \{2, 3\}$
<code>s.difference_update(s2,...)</code>	$s -= s2 - \dots$	<code>s1.difference_update(s2)</code> $\#s1 = \{1\}$
<code>s.symmetric_difference_update(s2)</code>	$s^{\wedge} = s2$	<code>s1.symmetric_difference_update(s2)</code> $\#s1 = \{1, 4\}$

可变集合set的方法2

■ set自有方法

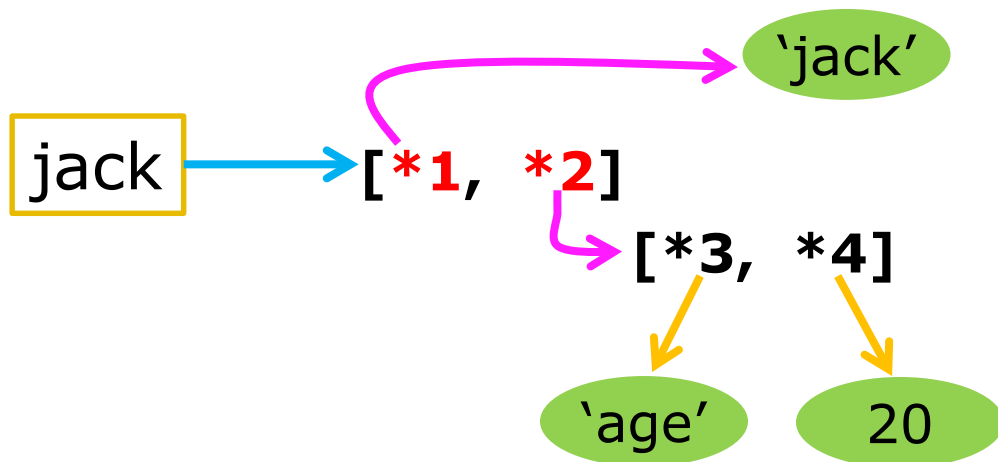
`s1={1,2,3}; s2={2,3,4}`

方法	说明	示例
<code>s.clear()</code>	清空	<code>s1.clear()</code> <code>#s1=set()</code>
<code>s.add(x)</code>	把对象x添加到s中	<code>s1.add('a')</code> <code>#s1={1,2,3,'a'}</code>
<code>s.remove(x)</code>	从s中删除x，如果x不存在，则导致KeyError	<code>s1.remove(1)</code> <code>#s1={2,3}</code> <code>s1.remove(4)</code> <code>#KeyError: 4</code>
<code>s.discard(x)</code>	从s中删除x，如果x存在的话	<code>s1.discard(1)</code> <code>#s1={2,3}</code> <code>s1.discard(4)</code> <code>#s1={1,2,3}</code>
<code>s.pop()</code>	随机弹出一个元素，如果s为空，则KeyError	<code>s1.pop()</code> <code>#输出1, s1={2,3}</code>

Python中列表、元组等存储机制

- 列表，元组，字典，集合等存储的不是成员对象本身，而是成员对象的引用。

```
>>> jack=['jack', ['age',20]]
```



拷贝

- 拷贝的方式有多种，如copy方法能实现拷贝（list, dict, set/frozenset）

```
>>> lst1=[1,2,3,1]
>>> lst2=lst1.copy()
>>> print(lst2, id(lst1), id(lst2))
[1, 2, 3, 1] 58272136 58271816
```

```
>>> s1=frozenset([1,1,2,2])
>>> s2=s1.copy()
>>> print(s2, id(s1), id(s2))
frozenset({1, 2}) 56035816 56035816
```

拷贝3

```
>>> jack=['jack', ['age', 20]]  
>>> tom=jack[:]  
>>> print(tom, id(tom), id(jack))  
['jack', ['age', 20]] 52041352 52040904
```

```
>>> tom[0]='tom'  
>>> print(tom, jack)  
['tom', ['age', 20]] ['jack', ['age', 20]]
```

```
>>> tom[1][1]=25  
>>> print(tom, jack)  
['tom', ['age', 25]] ['jack', ['age', 25]]
```

深拷贝

- 使用`copy`标准模块的`deepcopy`函数可复制出互不干涉的对象。

```
>>> import copy
>>> tom=copy.deepcopy(jack)
>>> tom[0]='tom'
>>> tom[1][1]=25
>>> print(jack,tom)
['jack', ['age', 20]] ['tom', ['age', 25]]
```